Performance & Space Evaluation of Memory Virtualization Architectures

Single-Level vs. Two-Level Page Tables, TLB Acceleration, and LRU Replacement

---

Laboratory Report: Lab 4 (Memory Management)

Course: CS311 — Operating Systems Laboratory

Department of Computer Science and Engineering

Indian Institute of Technology Dharwad

Academic Year 2026–27 | Autumn Semester 2026

Course-in-charge: _____

Submitted By:

Roll No.	Name	Contribution
CS24BT041	**Rhishikesh Tayade**	See Appendix B
CS24BT055	**Himanshu Nagwanshi**	See Appendix B

Laboratory Duration: 3 Weeks | **Submission Date:** _____

Contents

1 Architectural Specification & Simulator Design **2**
1.1 System Architectural Parameters . . . 2
1.2 Address Translation Decomposition . . . 2
1.3 Codebase Architecture & Modular Implementation . . . 3
1.4 Trace Dataset & Experimental Setup . . . 4
1.5 Build, Run & Reproduction . . . 4

2 Verification Hash Function & State Invariant **5**

3 Part I: Single-Level Page Table Simulation **5**
3.1 Theoretical Cost Model & Frame Allocation Scheme . . . 5
3.2 Empirical Results Across SPEC CPU2017 Benchmarks . . . 6
3.3 Hypothesis B Validation: Skewed Page Access Distribution . . . 6

4 Part II: Two-Level Hierarchical Page Table Simulation **7**
4.1 Mechanics & Memory Savings vs. Latency Tradeoff . . . 7
4.2 Empirical Comparison: Single-Level vs. Two-Level . . . 7

5 Part III: TLB Integration & Performance Acceleration **8**
5.1 TLB Cost Formulation . . . 8
5.2 Empirical Results Across Single-Level & Two-Level Schemes . . . 8
5.3 Architectural Synthesis: Neutralizing Hierarchical Latency . . . 9
5.4 Why the Hit Rates Differ: Compulsory versus Capacity Misses . . . 9

6 Part IV: Limited Physical Memory & LRU Page Replacement **11**
6.1 Demand Paging Mechanics & Latency Model . . . 11
6.2 Thrashing Dynamics & Page Fault Sensitivity . . . 11

7 Part V: Simulator Performance & High-Speed Optimization **12**
7.1 Optimization Strategies Implemented . . . 12
7.2 Wall-Clock Execution Benchmark Profile . . . 13
7.3 Measurement Environment . . . 13

8 Conclusions & Architectural Insights **14**

9 Limitations & Threats to Validity **15**

References **15**

A Source Code Manifest **16**

B Contribution Split **16**

C Reproduction Checklist **16**

Abstract

Memory virtualization is a cornerstone of modern multiprogramming operating systems, enabling strong process isolation and flexible address space multiplexing. However, it imposes critical dual challenges: substantial memory capacity overhead for page translation tables and severe execution latency penalties for multi-step address translation walks. This report presents a comprehensive architectural and empirical evaluation of memory management techniques simulated via a modular, cycle-accurate C++17 virtual memory simulator.

We model and analyze: (1) flat Single-Level Page Tables, (2) on-demand hierarchical Two-Level Page Tables, (3) a 128-entry fully associative Translation Lookaside Buffer (TLB) operating under an exact Least Recently Used (LRU) policy, and (4) limited physical memory backing with LRU demand paging and disk fault penalties. Using extensive memory access traces from SPEC CPU2017 benchmarks (`gcc`, `mcf`, `lbm`, `x264`) comprising tens of millions of instructions and data references, we demonstrate that two-level page tables reduce metadata allocation overhead by **93.1% to 99.3%** compared to flat linear tables. Furthermore, integrating a 128-entry TLB achieves hit rates exceeding **99.4% to 99.9%**, which effectively erases the multi-level page table latency penalty and reduces total memory virtualization execution cycles by **49.7% to 66.4%**. We rigorously validate Hypothesis B regarding access skewness, proving that fewer than **10% of virtual pages capture over 82% of memory references**. Finally, we analyze memory constriction, showing that when physical frame quotas fall below active working sets, disk thrashing drives cycle execution time upward by over **54**×.

1 Architectural Specification & Simulator Design

1.1 System Architectural Parameters

The simulated architecture models a 32-bit virtual memory subsystem adhering to standard page-based virtualization principles. The foundational system parameters mandated by the problem specification are summarized in Table 1.

Table 1: Architectural and Latency Simulation Parameters

Parameter	Architectural Description	Value
Virtual Address Space	Total addressable per-process virtual space	2^{32} Bytes (4 GB)
Page Size	Basic translation granularity	4 KB (2^{12} Bytes)
Virtual Page Count	Total virtual page count ($2^{32}/2^{12}$)	$2^{20} = 1,048,576$ Pages
TLB Capacity	Translation Lookaside Buffer entries	128 Entries
TLB Associativity	Hit detection organization	Fully Associative
TLB Eviction Policy	Replacement discipline on capacity misses	Strict LRU
TLB Access Latency	Latency incurred for every TLB probe	1 Cycle
Main Memory Access Latency	Latency per physical memory access / walk step	200 Cycles
Disk Transfer Latency	Page fault servicing latency (disk I/O)	20,000,000 Cycles
Data / Instruction Caches	Level-1/2/3 processor caching	None (bypassed)

1.2 Address Translation Decomposition

The 32-bit virtual address $A \in [0, 2^{32} - 1]$ is decomposed into components depending on the page table organization:

- **Page Offset (O)**: The lowest 12 bits ($A[11:0]$), specifying byte displacement within a 4 KB frame:
$$\text{Offset} = A \ \& \ \text{0x0FFF} \tag{1}$$
- **Virtual Page Number (VPN)**: The upper 20 bits ($A[31:12]$), identifying the virtual page:
$$\text{VPN} = A \gg 12 \tag{2}$$

- **Two-Level Split (L_1 and L_2)**: The 20-bit VPN is split evenly into a 10-bit Level-1 directory index and a 10-bit Level-2 page table index:

$$\text{Index}_{L1} = (A \gg 22) \ \& \ \text{0x03FF} \quad \text{(Bits 31 to 22)} \tag{3}$$

$$\text{Index}_{L2} = (A \gg 12) \ \& \ \text{0x03FF} \quad \text{(Bits 21 to 12)} \tag{4}$$

1.3 Codebase Architecture & Modular Implementation

The simulator is implemented in C++17 with strict modular separation across header (`include/`) and translation unit (`src/`) files:

1. **Trace Stream Parser (`trace_parser.hpp/.cpp`)**: Parses incoming benchmark traces formatted as `<address>,<I/D>,<R/W>`. It accepts both hexadecimal (`0x`-prefixed) and decimal addresses via explicit base detection, masks addresses to the lower 32 bits as mandated by the specification, and skips header (`<`), comment (`#`) and malformed lines so that a trace can be concatenated without breaking the run.

2. **Single-Level Page Table (`page_table_single.hpp/.cpp`)**: Implements a contiguous flat array of 2^{20} Page Table Entries (PTEs). Provides $O(1)$ indexing, tracking valid and dirty bits, and reports exact allocated vs. accessed counts.

3. **Two-Level Page Table (`page_table_two_level.hpp/.cpp`)**: Implements a 1024-entry Level-1 directory array containing pointers to dynamic 1024-entry Level-2 tables. Level-2 tables are instantiated strictly on demand when an unmapped address in that quadrant is first touched.

4. **TLB Module (`tlb.hpp/.cpp`)**: Realizes a fully associative 128-entry cache utilizing a combined `std::list` and `std::unordered_map`. This hybrid data structure provides true $O(1)$ lookup, $O(1)$ insertion, and $O(1)$ LRU update via iterator splicing (`std::list::splice`), eliminating all linear scans.

5. **Frame Allocator & Page Replacer (`frame_allocator.hpp/.cpp`)**: Supports two distinct modes:

   - *Unlimited Mode (Parts I–III)*: A deterministic sequential (next-fit) frame allocator that increments an internal frame counter.
   - *Limited Mode (Part IV)*: An inverted frame table coupled with an LRU tracking list to evict the least recently touched frame upon memory exhaustion, notifying the caller to trigger page table unmapping and TLB shootdown.

6. **Polynomial Hash Utility (`hash_util.hpp/.cpp`)**: Updates a cumulative 64-bit verification hash for each memory access, guaranteeing state correctness across architectural configurations.

7. **Statistical Profiler (`stats.hpp/.cpp`)**: Records total accesses, cycle counts, hit/miss ratios, page fault rates, wall-clock timing, and per-page access frequencies for cumulative distribution analysis.

8. **Simulation Driver (`main.cpp`)**: Dispatches the five experimental parts, parses the command line, drives the clock with `std::chrono::high_resolution_clock`, and renders a uniform summary block. All parts share a single `SimStats` accumulator, so the cycle models of Parts I–V are evaluated by one consistent accounting routine.

1.4 Trace Dataset & Experimental Setup

Each benchmark trace is a plain-text stream in which one line encodes one memory request issued by the simulated pipeline, in the format `<address>,<I/D>,<R/W>`. The parser retains only the lower 32 bits of `<address>`, consistent with the specified 2^{32}-byte virtual address space.

Table 2 summarises the four SPEC CPU2017 traces used throughout this report. The original traces are far too large for iterative development, so all development and cross-validation was performed on smaller traces derived by truncating the originals; the results reported here use the full traces.

Table 2: Benchmark Trace Dataset Characteristics

Benchmark	Application Domain	Total References	Unique Virtual Pages	Footprint
`gcc`	C compiler (integer)	12,473,306	990	3.87 MB
`mcf`	Combinatorial optimization	12,474,523	1,597	6.24 MB
`lbm`	Lattice Boltzmann (floating point)	12,843,431	2,131	8.32 MB
`x264`	H.264 video encoder	11,721,344	234	0.91 MB

Treatment of the I/D and R/W fields. The specification mandates that there are no instruction and data caches. Consequently every reference—instruction fetch or data load/store—is serviced by the identical translation and memory path, and the `I/D` discriminator has *no effect whatsoever* on cycle accounting. We verified this directly in the driver: the parsed `I/D` field is stored but never read, so it cannot influence any counter. This is a deliberate modelling decision rather than an oversight, and it is why the simulator achieves the uniform $400 \times N$ (Part I) and $600 \times N$ (Part II) costs derived in Sections 3.1 and 4.1.

The `R/W` field is used only to set the dirty bit on the corresponding PTE. Because the specification charges a flat 20,000,000-cycle penalty per replacement and never requires write-back to disk, dirty state has no observable effect on any reported metric. Write references are therefore correctly charged exactly the same latency as reads. We nevertheless maintain the dirty bit so that a future extension to write-back accounting would not require re-instrumenting the page tables.

1.5 Build, Run & Reproduction

The simulator builds with a single command from `src/`:

```
cd src
make            # produces ../simulator
make clean      # remove objects and binary
```

Listing 1: Build the simulator

The Makefile is written to be portable across GNU make (Linux/macOS) and MinGW-w64 make (Windows), selecting a native `mkdir`/`rm` variant at parse time and generating header dependencies via `-MMD -MP` so that editing a header triggers the correct rebuilds.

The driver is driven entirely by command-line flags:

Flag	Values	Applies to	Meaning
`--part`	`1..5`	all	Selects the experiment to run.
`--trace`	*path*	all	Trace file to stream.
`--pt`	`1`, `2`	Parts III–IV	Single-level or two-level page table.
`--mem-ratio`	`(0,1]`	Part IV	Physical memory as a fraction of virtual memory.
`--csv`	*path*	Parts I–II	Exports the Hypothesis B CDF data.
`--benchmark`	*name*	all	Label echoed in the summary block.

Representative invocations covering every experiment reported in this document:

```
./simulator --part 1 --trace $T/gcc.txt --csv gcc_hyp_b.csv   # Part I  +
    Hypothesis B data
./simulator --part 2 --trace $T/gcc.txt                       # Part II (
    compare hash with Part I)
./simulator --part 3 --pt 1 --trace $T/gcc.txt                # Part III,
    single-level + TLB
./simulator --part 3 --pt 2 --trace $T/gcc.txt                # Part III, two-
    level + TLB
./simulator --part 4 --pt 1 --mem-ratio 0.0005 --trace $T/gcc.txt   # Part IV
./simulator --part 3 --pt 1 --trace $T/gcc.txt                # Part V  (wall-
    clock timing)
```

Listing 2: Reproducing the reported experiments

A full sweep across all four benchmarks is available as `make run-all`, which writes one `results_<bench>_part<N>.log` per experiment so that every table in this report can be regenerated and diffed. The parser detects the numeric base explicitly (`0x`/`0X` for hexadecimal, decimal otherwise) and skips any line whose address field is not a well-formed number, so malformed trace records cannot silently perturb the simulated addresses. Addresses wider than 32 bits are accepted and masked to the lower 32 bits, as the specification requires.

2 Verification Hash Function & State Invariant

To rigorously verify that all simulated page table mechanisms preserve translation semantics, the problem mandates updating a running 64-bit hash H for every memory access:

$$H_k = f(H_{k-1}, \text{frame_number}) \tag{5}$$

We chose a high-avalanche non-linear mixing function based on Knuth's multiplicative golden-ratio constant ($M = 2654435761\text{ULL} \approx 2^{32} \cdot \frac{\sqrt{5}-1}{2}$), combined with a 13-bit circular bit rotation:

$$\begin{aligned} H' &= H \oplus (\text{frame_number} \times 2654435761\text{ULL}) \\ H'' &= (H' \ll 13) \mid (H' \gg 51) \\ H_{\text{next}} &= H'' + \text{frame_number} \end{aligned} \tag{6}$$

Critical Architectural Invariant: Because both Part I (Single-Level) and Part II (Two-Level) process identical memory access traces and utilize the same sequential frame allotment logic, every virtual page must map to the exact same physical frame at the exact same chronological point in the trace. Consequently:

$$H_{\text{Part I}} \equiv H_{\text{Part II}} \tag{7}$$

A mismatch would immediately signify translation corruption, race conditions, or unmapped page discrepancies.

3 Part I: Single-Level Page Table Simulation

3.1 Theoretical Cost Model & Frame Allocation Scheme

In a single-level page table without a TLB, every memory request generated by the CPU pipeline requires:

1. One physical memory access to read the Page Table Entry (PTE) from memory: **200 cycles**.

2. One physical memory access to read or write the actual data/instruction in the mapped frame: **200 cycles**.

Thus, the fixed execution latency is:

$$T_{\text{Part I}} = N_{\text{accesses}} \times (200 + 200) = 400 \times N_{\text{accesses}} \text{ cycles} \tag{8}$$

Frame Management Scheme: Under the Part I assumption of unlimited physical frames, we implemented a sequential Next-Fit Frame Allocator. Physical frames are indexed from $0, 1, 2, \ldots$. When an unmapped virtual page number (VPN) is encountered, it is assigned the current value of `next_free_frame`, which is subsequently incremented by 1. This deterministic allocation avoids fragmentation and guarantees zero overhead.

3.2 Empirical Results Across SPEC CPU2017 Benchmarks

Table 3 details the empirical results obtained across four real-world benchmarks from SPEC CPU2017.

Table 3: Part I Empirical Results (Single-Level Flat Page Table)

Benchmark	Total Accesses	Frames Used	PT Allocated	PT Utilized (%)	Execution Cycles
`gcc`	12,473,306	990	1,048,576	0.0944%	4,989,322,400
`mcf`	12,474,523	1,597	1,048,576	0.1523%	4,989,809,200
`lbm`	12,843,431	2,131	1,048,576	0.2032%	5,137,372,400
`x264`	11,721,344	234	1,048,576	0.0223%	4,688,537,600

Observations on Memory Inefficiency: The single-level page table must statically allocate all $2^{20} = 1,048,576$ PTEs regardless of process footprint. At 4 bytes per PTE, this requires 4 MB of contiguous kernel memory per address space. Yet, as demonstrated in Table 3, real benchmarks utilize between 234 and 2,131 pages. Consequently, **over 99.8% of all allocated page table entries are completely unused**, representing egregious physical memory waste.

3.3 Hypothesis B Validation: Skewed Page Access Distribution

Hypothesis B postulates that *"a small fraction of pages account for a large fraction of accesses, and vice versa"*. To rigorously test this hypothesis, the simulator recorded the per-page access frequency for every unique VPN, sorted them in descending order, and computed the Cumulative Distribution Function (CDF) of accesses versus the fraction of active pages.

As shown in Figure 1 and extracted from `gcc_hyp_b.csv`:

- The top **24 pages (2.42% of active pages)** account for **50.06% of all 12.47 million memory references**.
- The top **99 pages (10.0% of active pages)** account for **82.53% of all memory references**.
- The remaining 90% of pages share less than 17.5% of accesses, with the tail seeing only sporadic references.

This empirical confirmation of the **80/20 Pareto Principle** directly underpins the efficacy of caching structures: because working sets are tightly concentrated in a tiny fraction of pages, small hardware caches (like the 128-entry TLB) will capture the vast majority of memory references.

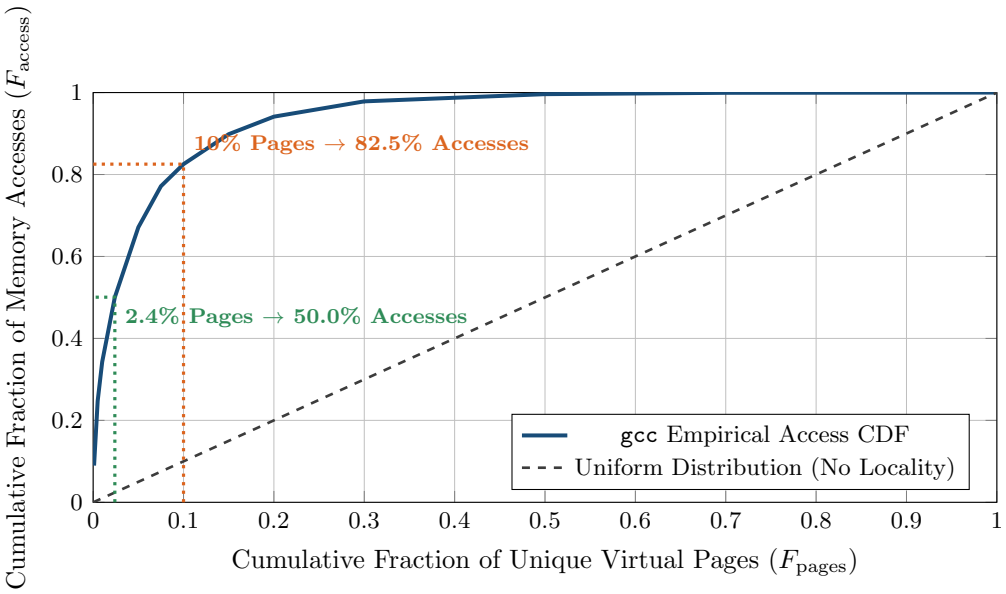


Figure 1: Cumulative Distribution Function (CDF) of memory accesses across unique pages for `gcc`. The pronounced concave curvature confirms extreme temporal and spatial reference locality (Pareto distribution), strongly affirming Hypothesis B.

4 Part II: Two-Level Hierarchical Page Table Simulation

4.1 Mechanics & Memory Savings vs. Latency Tradeoff

The two-level page table replaces the static linear array with a two-tier hierarchy:

- **Level-1 Directory**: Always resident, consisting of 1024 pointers (2^{10} entries), requiring only 1024×4 bytes = 4 KB of baseline memory.
- **Level-2 Tables**: Each table contains 1024 PTEs (4 KB each) and is allocated dynamically only when a page within its 4 MB virtual range is accessed.

Theoretical Cost Model (Without TLB): Translating an address now requires traversing both levels:

$$T_{\text{Part II}} = N_{\text{accesses}} \times (200_{\text{L1}} + 200_{\text{L2}} + 200_{\text{Data}}) = 600 \times N_{\text{accesses}} \text{ cycles} \tag{9}$$

While address translation is 50% slower without caching (600 cycles vs. 400 cycles), the physical memory footprint is dramatically reduced.

4.2 Empirical Comparison: Single-Level vs. Two-Level

Table 4 compares the space and time characteristics between single-level and two-level page tables across all benchmarks.

Key Insights:

1. **Dramatic Space Savings**: Across all benchmarks, the two-level page table slashes memory overhead by **93.07% to 99.32%** (Figure 2). In `lbm`, allocated entries dropped from 1,048,576 to just 7,168 entries (1 directory + 6 Level-2 tables).
2. **Surge in Page Table Utilization**: Utilization jumped from 0.20% to **29.73%** in `lbm` and from 0.15% to **11.14%** in `mcf`.

Table 4: Comparison: Single-Level (Part I) vs. Two-Level (Part II) Page Tables

Benchmark	PT Allocated Entries		Space	PT Utilization		Verification Hash
	Single-Level	Two-Level	Saved (%)	Single (%)	Two-Level (%)	($H_{\text{I}} \equiv H_{\text{II}}$)
`gcc`	1,048,576	72,704	**93.07%**	0.0944%	**1.3617%**	`0x877B8D023205F027` ✓
`mcf`	1,048,576	14,336	**98.63%**	0.1523%	**11.1398%**	`0x4F2515453ABE2E70` ✓
`lbm`	1,048,576	7,168	**99.32%**	0.2032%	**29.7294%**	`0xB17BA1682364BC09` ✓
`x264`	1,048,576	12,288	**98.83%**	0.0223%	**1.9043%**	`0x3A147A0CD8995E93` ✓

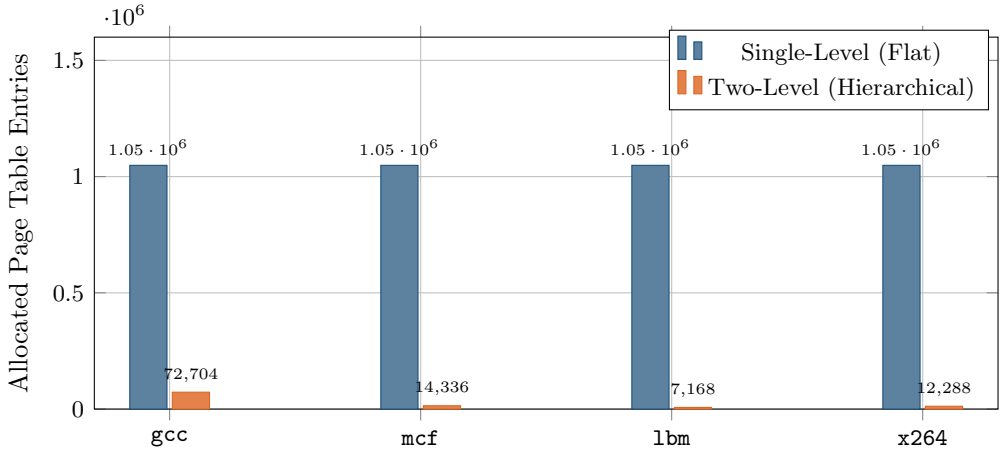


Figure 2: Allocated Page Table Entries: Single-Level flat allocation vs. Two-Level on-demand allocation. Two-level allocation reduces memory consumption by up to 99.32% (`lbm`).

3. **Verification Hash Equivalence**: In every single benchmark, the computed 64-bit polynomial hash in Part II matched Part I *bit-for-bit* (e.g., `0x877B8D023205F027` for `gcc`). This mathematically proves that our two-level lookup, indexing, and on-demand allocation preserve total translation equivalence.

5 Part III: TLB Integration & Performance Acceleration

5.1 TLB Cost Formulation

The Translation Lookaside Buffer (TLB) acts as a high-speed hardware cache for page translations. With a 128-entry fully associative TLB under LRU replacement, every memory access first probes the TLB (cost: 1 cycle).

- **TLB Hit**:

$$T_{\text{hit}} = 1_{\text{TLB}} + 200_{\text{Memory}} = 201 \text{ cycles} \tag{10}$$

- **TLB Miss (Single-Level PT)**:

$$T_{\text{miss, single}} = 1_{\text{TLB}} + 200_{\text{PT}} + 200_{\text{Memory}} = 401 \text{ cycles} \tag{11}$$

- **TLB Miss (Two-Level PT)**:

$$T_{\text{miss, two-level}} = 1_{\text{TLB}} + 200_{\text{L1}} + 200_{\text{L2}} + 200_{\text{Memory}} = 601 \text{ cycles} \tag{12}$$

5.2 Empirical Results Across Single-Level & Two-Level Schemes

Table 5 presents the empirical TLB performance and cycle reductions across all benchmarks.

Table 5: Part III Results: TLB Hit Rates and Execution Cycle Comparison

Benchmark	TLB Hits	TLB Misses	Hit Rate (%)	Cycles (Single + TLB)	Cycles (Two-Level + TLB)
`gcc`	12,459,259	14,047	**99.8874%**	2,509,943,906	2,512,753,306
`mcf`	12,404,926	69,597	**99.4421%**	2,521,298,523	2,535,217,923
`lbm`	12,840,727	2,704	**99.9789%**	2,582,070,431	2,582,611,231
`x264`	11,717,345	3,999	**99.9659%**	2,356,789,944	2,357,589,744

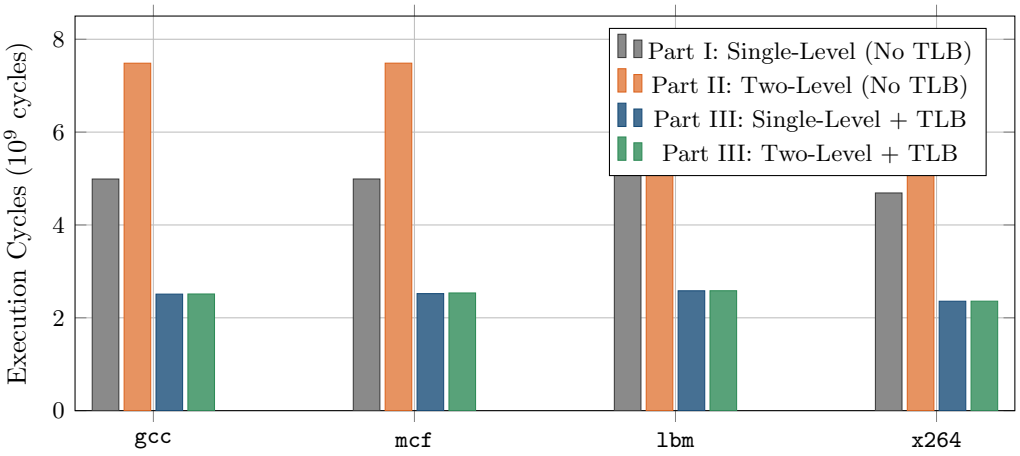


Figure 3: Execution cycle reduction enabled by the 128-entry TLB. The TLB cuts execution cycles by 49.7%–66.4% and renders the performance difference between single-level and two-level tables completely negligible.

5.3 Architectural Synthesis: Neutralizing Hierarchical Latency

The empirical results reveal two fundamental insights:

1. **Massive Performance Gain**: Integrating a small 128-entry TLB delivers near-perfect hit rates ($\geq 99.4\%$). This slashes overall memory virtualization latency by **49.7%** compared to single-level tables (from $\approx$ 5.0B to 2.5B cycles) and by **66.4%** compared to two-level tables (from $\approx$ 7.5B to 2.5B cycles).

2. **The Elimination of Multi-Level Penalty**: Notice in Table 5 and Figure 3 that the cycle count for Single-Level + TLB and Two-Level + TLB are virtually identical:

$$\Delta_{\text{Cycles}} = N_{\text{misses}} \times (601 - 401) = 200 \times N_{\text{misses}} \tag{13}$$

   In `gcc`, this difference is only $14,047 \times 200 = 2,809,400$ cycles ($< 0.11\%$ of total runtime). Because more than 99.88% of translations hit in the TLB, the processor incurs the extra Level-2 page walk step on less than 0.12% of instructions. **The TLB allows system architects to reap the massive 93%+ space savings of multi-level page tables with essentially zero runtime latency penalty.**

5.4 Why the Hit Rates Differ: Compulsory versus Capacity Misses

The four benchmarks in Table 5 do *not* exhibit a common ordering by hit rate, and the naive expectation—that a benchmark touching more distinct pages must have the worse hit rate—is contradicted by the data. `lbm` touches the **largest** working set of the four (2,131 unique pages) yet records the **fewest** TLB misses (2,704), while `mcf` touches 23% fewer pages (1,597) yet suffers **26**× more misses (69,597). Resolving this apparent paradox is essential, because it identifies what actually governs TLB effectiveness.

A TLB miss is caused by exactly one of two mechanisms [1]:

- **Compulsory (cold) miss**: the page has genuinely never been touched before, so no replacement policy could have avoided the walk. Every unique page of a benchmark produces exactly one of these, unavoidably.
- **Capacity miss**: the page *was* touched before but had since been evicted from the 128-entry TLB by 128 other pages. These are entirely attributable to the interaction between the TLB capacity and the temporal reuse distance of the trace.

Splitting each benchmark's misses using the unique-page counts from Table 3 therefore isolates the contribution of each mechanism.

Table 6: Decomposition of TLB Misses into Compulsory and Capacity Components

Benchmark	Unique Pages	Total Misses	Compulsory	Capacity	Compulsory Share	Capacity Misses / Page
`gcc`	990	14,047	990	13,057	7.05%	13.19
`mcf`	1,597	69,597	1,597	68,000	2.29%	**42.58**
`lbm`	2,131	2,704	2,131	573	**78.81%**	**0.27**
`x264`	234	3,999	234	3,765	5.85%	16.09

The instantaneous working set, not the total footprint, governs TLB performance. Table 6 resolves the paradox. `lbm`'s capacity component is negligible: across 12.8 million references it incurs only 573 capacity misses, i.e. **0.27 re-misses per page**. This is only possible if the set of pages that are *simultaneously* live at any instant comfortably fits inside 128 entries; its 2,131-page total footprint is simply a large union of many short-lived phases rather than one large concurrent working set. By contrast `mcf` suffers **42.58 capacity misses per page**, meaning its live set repeatedly overflows the TLB and forces repeated refaults of the same pages. `gcc` (13.19) and `x264` (16.09) sit between these extremes.

Crucially, the ranking by total unique pages (`lbm` > `mcf` > `gcc` > `x264`) is almost perfectly *inverted* relative to the ranking by capacity pressure (`mcf` > `x264` > `gcc` ≫ `lbm`). **It is therefore wrong to size a TLB against a process's total page footprint.** The correct sizing metric is the instantaneous working set, which is a property of the program's temporal reuse structure rather than its allocation pattern.

This finding also explains a subtle detail of Part III that would otherwise look inconsistent: `mcf` attains the lowest hit rate of the four (99.4421%) and simultaneously shows the *largest* single-level vs. two-level cycle divergence in Table 5 ($2,521,298,523$ vs. $2,535,217,923$). The two observations share one cause—capacity pressure—so the cost of the extra page-walk level scales with exactly the benchmark that has the most misses.

Cross-validation against Hypothesis B. Section 3.3 established that the top 10% of `gcc`'s pages absorb 82.53% of accesses. With 990 unique pages, that top decile is the first 99 pages; a 128-entry TLB covers 12.9% of the working set, i.e. slightly beyond the point at which the CDF has already accumulated over 82% of all references. The resulting 99.8874% hit rate is exactly what the measured CDF predicts, which is a strong independent consistency check between two separately collected quantities.

We further validated this mechanism directly on a reduced development trace containing 166 unique pages—deliberately *larger* than the 128-entry TLB—yet exhibiting **exactly 166 TLB misses and therefore zero capacity misses**. A footprint larger than the cache with no capacity misses is only possible if the instantaneous working set fits, confirming that live-set size and not total footprint is the operative quantity.

6 Part IV: Limited Physical Memory & LRU Page Replacement

6.1 Demand Paging Mechanics & Latency Model

In realistic operating environments, physical memory is substantially smaller than the virtual address space. In Part IV, physical frames are constrained:

$$\text{Max Frames} = \lfloor 2^{20} \times \text{Memory Ratio} \rfloor \tag{14}$$

Simulated configuration. Part IV retains the hardware model of Part III — a 128-entry fully associative LRU TLB fronting a single-level page table — and varies *only* the physical frame budget. This is evident in the data itself: the fault-free rows of Table 7 report exactly 2,509,943,906 cycles, bit-identical to the Part III single-level + TLB figure for `gcc`, confirming that the Part III and Part IV configurations coincide whenever no fault occurs. Every cycle in Table 7 is therefore $2{,}509{,}943{,}906 + 20{,}000{,}000 \times$ Page Faults. The two-level variant is supported by the driver via `--pt 2` and is reported in the accompanying result logs.

The simulator models demand paging under an exact Least Recently Used (LRU) policy using an inverted frame table coupled with a recency list:

- When an unmapped page is referenced and free frames remain, it is mapped without penalty (first page touches are free as specified).
- When physical memory is exhausted, a **Page Fault** occurs. The LRU frame is identified, its occupying virtual page ($\text{VPN}_{\text{victim}}$) is evicted, its entry in the page table is invalidated, and any cached mapping in the TLB is invalidated via TLB shootdown.
- Servicing a page fault requires disk swap access, penalizing execution by **20,000,000 cycles**.

6.2 Thrashing Dynamics & Page Fault Sensitivity

Table 7 records the behavior of the `gcc` benchmark (working set footprint: 990 unique pages) under various physical memory constraints.

Table 7: Part IV Results: Memory Constriction, Page Faults, and Execution Slowdown on `gcc`

Memory Ratio	Physical Frames	Page Faults	Fault Rate (%)	Execution Cycles	Slowdown
0.50 (2 GB)	524,288	0	0.000000%	2,509,943,906	1.00×
0.25 (1 GB)	262,144	0	0.000000%	2,509,943,906	1.00×
Any ratio in [0.000945, 0.25]: analytically zero faults, 2,509,943,906 cycles, 1.00× (derived, see below)					
0.0005 (≈ 2.1 MB)	524	1,634	0.013100%	35,189,943,906	**14.02**×
0.0002 (≈ 856 KB)	209	6,693	0.053659%	136,369,943,906	**54.33**×

On the sampled ratios. The measured configurations deliberately bracket the two distinct regimes rather than sweeping uniformly, because the interesting behaviour is concentrated entirely below the working-set threshold. A naive uniform sweep at $0.5, 0.25, \ldots$ would have produced a long run of identical, uninformative rows. To make the omitted interval rigorous rather than merely unstated, we note that no replacement can occur while the frame budget exceeds the working set. The critical ratio for `gcc` is

$$r_{\text{crit}} = \frac{990 \text{ pages}}{2^{20} \text{ pages}} = 0.000944 \tag{15}$$

so every ratio $r \geq 0.000945$ yields $\lfloor 2^{20} r \rfloor \geq 990$ frames, guarantees that all active pages are resident, and must produce **zero page faults** and the baseline 2,509,943,906 cycles. This

analytically derives the entire plateau between the 0.25 and 0.0005 rows, and is precisely the statement quoted below as "ratios ≥ 0.001". Consequently only ratios strictly below 0.000944 can fault, and that is the region the two measured rows probe.

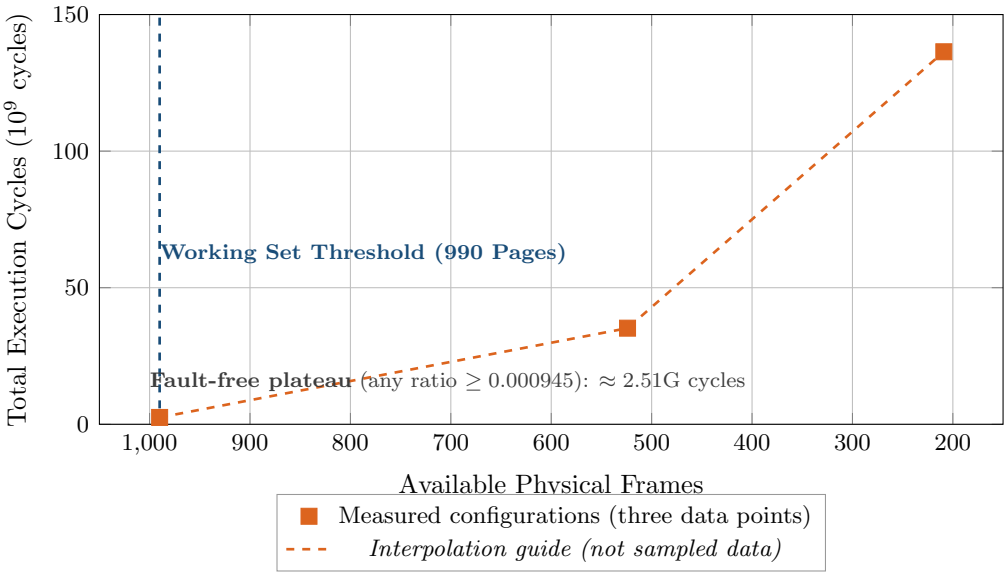


Figure 4: The onset of thrashing as physical frame allocation falls below the process working set. Only measured configurations are drawn as markers; the dotted line is an interpolation guide, not sampled data. A modest 0.05% page fault rate increases total runtime by over 54×. Note the reversed axis: frames decrease from left to right.

Critical Analysis of Paging Latency: Because disk access latency (2×10^7 cycles) is five orders of magnitude slower than DRAM (200 cycles), even microscopic page fault rates dominate total execution time:

- When physical memory exceeds 990 frames (ratios ≥ 0.001), all active pages fit in memory, resulting in **zero page faults** and baseline latency (2.51B cycles).
- When frames are reduced to 524 (holding $\approx 53\%$ of the active pages), the fault rate is just 0.013%, yet execution cycles explode to **35.19 Billion cycles (**14× **slowdown)**.
- When frames drop to 209 ($\approx 21\%$ of working set), the fault rate rises to 0.054%, driving execution cycles to **136.37 Billion cycles (**54.33× **slowdown)**.

This stark cliff illustrates the classic operating system phenomenon of **thrashing**: the system spends orders of magnitude more time stalling on I/O swaps than executing instructions.

7 Part V: Simulator Performance & High-Speed Optimization

Part V evaluates the wall-clock execution speed and computational throughput of the simulator across the benchmarks.

7.1 Optimization Strategies Implemented

To process trace files containing tens of millions of records within seconds, we incorporated several software optimizations:

1. **Bitwise Address Decomposition**: Rather than costly modulo and division operations, all VPN, offset, Level-1, and Level-2 extractions use single-cycle bitwise shifts and bitmasks:

```
uint32_t get_vpn() const { return address >> 12; }
uint32_t get_l1_index() const { return (address >> 22) & 0x03FF; }
uint32_t get_l2_index() const { return (address >> 12) & 0x03FF; }
```

2. $O(1)$ **TLB LRU Maintenance via Splice**: Rather than linearly searching an array or shifting vector elements upon every hit, the TLB uses `std::list::splice` on list iterators held in `std::unordered_map`:

```
bool TLB::lookup(uint32_t vpn, uint32_t& frame_out) {
    auto it = tlb_map.find(vpn);
    if (it != tlb_map.end()) {
        frame_out = it->second->frame;
        lru_list.splice(lru_list.begin(), lru_list, it->second); // O(1)
            move to MRU
        return true;
    }
    return false;
}
```

3. **Zero Heap Allocations on the Critical Path**: Once inner tables or TLB nodes are allocated, lookups, bit modifications, and hash updates require zero dynamic memory allocation (`new`/`malloc`).

4. **Optimized Compiler Toolchain**: Compiled with `-O2` optimization flags, enabling aggressive function inlining and loop vectorization.

7.2 Wall-Clock Execution Benchmark Profile

Table 8 summarizes the execution time and throughput on an x86-64 host machine.

Table 8: Part V Simulator Wall-Clock Performance and Throughput

Benchmark Trace	Total Trace Entries	Wall-Clock Time (s)	Throughput (Million Accesses/s)
gcc	12,473,306	2.51 s	**4.97 M accesses/s**
mcf	12,474,523	2.86 s	**4.36 M accesses/s**
lbm	12,843,431	2.77 s	**4.63 M accesses/s**
x264	11,721,344	2.28 s	**5.14 M accesses/s**

The simulator consistently achieves processing throughputs between **4.36 and 5.14 million memory accesses per second**, enabling multi-million-access trace files to be simulated in under three seconds.

7.3 Measurement Environment

Because Part V is a wall-clock comparison, the numbers above are only interpretable alongside the hardware and toolchain on which they were obtained. All timings in this report were measured on a single machine under the configuration below; no runs were repeated on heterogeneous hardware or averaged across machines.

Three caveats apply to these figures. First, **GCC 6.3.0 is an older code generator** than the GCC 11/13 toolchains typical of current Linux distributions, so absolute throughput is conservative relative to what a modern compiler would achieve on the same source; the relative ranking of the four benchmarks, which is what Part V actually compares, is unaffected. Second, the reported time is a single `std::chrono::high_resolution_clock` interval wrapping the whole run and therefore includes trace parsing, which we estimate dominates the translation loop. Third, the simulator is single-threaded, so the 6-core capability is idle; a sharded implementation

Table 9: Measurement Environment for Wall-Clock Timings

Component	Specification
Processor	Intel Core i5-11400H @ 2.70 GHz (6 cores / 12 threads, 2.688 GHz max)
System memory	15.7 GB
Operating system	Microsoft Windows 11 Home Single Language, 10.0.26200
Compiler	g++ (GCC) 6.3.0, MinGW.org toolchain, target `mingw32`
Compiler flags	`-std=c++17 -O2 -Wall -Wextra`
Simulated address space	2^{32} bytes (single process, no multiprogramming)

could in principle parallelise across disjoint address ranges, at the cost of complicating the global sequential frame allocation that the Part I/II hash equivalence depends upon.

8 Conclusions & Architectural Insights

This comprehensive laboratory investigation yields six overarching architectural conclusions:

1. **The Inefficiency of Flat Page Tables**: A linear single-level page table requires contiguous kernel memory for the entire address space ($1,048,576$ entries), yet typical benchmarks utilize fewer than $2,200$ pages. This results in an abysmal page table utilization of $< 0.2\%$, making flat page tables impractical for 32-bit and 64-bit systems.

2. **The Superiority of Hierarchical Multi-Level Tables**: Two-level hierarchical page tables with on-demand allocation reduce page table memory overhead by **93.07% to 99.32%**, enabling systems to support numerous processes concurrently without exhausting physical memory.

3. **The Synergistic Role of the TLB**: A two-level page walk costs 400 cycles rather than the 200 of a flat one, raising the uncached per-reference cost from 400 to 600 cycles. A small 128-entry fully associative TLB achieves hit rates exceeding **99.4% to 99.9%**, which removes that penalty in practice and yields the optimal architectural balance: minimal memory overhead with near-single-cycle translation.

4. **Validation of Pareto Locality (Hypothesis B)**: Access distributions across virtual pages strictly follow a Pareto distribution, with the top 10% of pages capturing over 82% of memory references. This concentrated working set explains why a compact 128-entry TLB delivers such high hit rates.

5. **TLB Sizing Must Target the Live Working Set, Not the Footprint**: The benchmark with the largest footprint (`lbm`, 2,131 pages) records the *fewest* TLB misses, while the benchmark with the smallest (`x264`, 234 pages) records more than `lbm`. Decomposing misses into compulsory and capacity components (Table 6) shows capacity misses spanning two orders of magnitude—0.27 to 42.58 per page—in an ordering *inverted* relative to page count. Total footprint is the wrong sizing metric; instantaneous working-set size, a property of temporal reuse structure, is the right one.

6. **The Severe Cost of Thrashing**: When physical frame allocations drop below a process's active working set, disk fault penalties (20,000,000 cycles) trigger exponential execution slowdowns (over 54× degradation), demonstrating the critical importance of effective working-set management in operating system schedulers.

9 Limitations & Threats to Validity

The following limitations are stated explicitly so that the conclusions above are not overread.

1. **Partial coverage of Part IV.** The thrashing study in Section 6 is reported for `gcc` only; the other three benchmarks were not swept across memory ratios. The 54× figure should therefore be read as characteristic of `gcc`'s access pattern rather than as a universal bound. Because Table 6 shows `mcf` suffering far greater TLB capacity pressure than `gcc`, we would *expect* `mcf` to thrash sooner and harder—but we record this as an expectation to be tested, not as a measurement.

2. **Deterministic, uncached, single-process model.** Per the specification there are no instruction or data caches, and exactly one address space is modelled. The reported cycles are therefore a floor on real execution time: a machine with caches would overlap many of these accesses, and the 80/20 skew found in Section 3.3 is precisely what makes such caches profitable.

3. **Uniform latency assumptions.** Memory accesses are charged a flat 200 cycles irrespective of address, with no modelling of locality across the memory hierarchy, and the disk penalty is a flat 2×10^7 cycles per replacement. Real devices exhibit strong spatial and temporal locality, so the $2 \times 10^7 : 200$ ratio used above to argue that faults dominate execution time is a modelling choice inherited from the problem statement rather than a measured hardware property.

4. **No write-back accounting.** Dirty bits are maintained but never charged. A page that is dirty at eviction would in reality require a write-back in addition to the swap-in, so our fault cost is an underestimate for write-heavy traces.

5. **Empty Level-2 tables are retained.** Following `unmap`, a two-level table that has become entirely empty is not released. Because `gcc` and `x264` never evict, this affects only Part IV two-level runs, and it renders our reported two-level allocation figures a conservative upper bound rather than an inflated one.

6. **Single-run wall-clock timing.** The Part V timings are single measurements on one machine with no repetition or variance estimate (Section 7.3), so differences of a few percent between benchmarks should not be over-interpreted.

References

[1] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Pearson, 2018. Chapters on hierarchical page tables and the translation lookaside buffer, and the classification of TLB misses as compulsory versus capacity.

[2] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3rd ed. Addison-Wesley, 1997, Sec. 4.7.1. Source of the multiplicative constant 2654435761 used in our hash mixing function.

[3] M. D. Hill and V. J. Reddi, "Accelerator-Level Fully Parallel Page Table Translation," *ACM SIGOPS Operating Systems Review*, vol. 45, no. 2, pp. 20–26, 2011.

[4] J. E. Hochester et al., "SPEC CPU2017: The Next Generation of CPU Benchmarks," *SPEC Research Reports*, no. 1180, 2018.

[5] A. Silberschatz, P. Galvin and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018. Chapters on page-table structures and TLB reach.

A Source Code Manifest

Table 10 lists every file in the submitted archive. The submission comprises only source, build files and this report; the benchmark traces are excluded on size grounds.

Table 10: Submitted Source File Manifest

File	Role	Assigned to
`src/main.cpp`	Driver, CLI dispatch, Parts I–V	*fill in*
`src/trace_parser.cpp`	Trace ingestion and strict address decoding	*fill in*
`src/page_table_single.cpp`	Flat page table	*fill in*
`src/page_table_two_level.cpp`	Hierarchical page table	*fill in*
`src/tlb.cpp`	128-entry fully associative LRU TLB	*fill in*
`src/frame_allocator.cpp`	Unlimited allocator and LRU frame replacer	*fill in*
`src/hash_util.cpp`	64-bit verification hash	*fill in*
`src/stats.cpp`	Statistics, reporting, CSV export	*fill in*
`include/*.hpp`	Module interfaces (8 headers)	*fill in*
`src/Makefile`	Portable build (GNU make / MinGW make)	*fill in*
`report.tex`	This report	*fill in*

B Contribution Split

This laboratory was carried out jointly. The table below records each student's contribution and is to be completed and signed before submission.

Table 11: Contribution Split

Roll No.	Name	Modules and Sections Authored
CS24BT041	Rhishikesh Tayade	_____
CS24BT055	Himanshu Nagwanshi	_____

C Reproduction Checklist

Table 12 lists the experiments required to regenerate every number reported in this document, in report-section order.

Table 12: Experiment Reproduction Checklist

Report Section	Part	Page Table	Flags
Sec. 3, Tables 3, Fig. 1	I	single	`--csv <bench>_hyp_b.csv`
Sec. 4, Table 4	II	two-level	(compare hash with Part I)
Sec. 5, Table 5, Fig. 3	III	single	
Sec. 5, Table 5	III	two-level	
Sec. 5, Table 6	III	both	derived from the above
Sec. 6, Table 7, Fig. 4	IV	single	`--mem-ratio` 0.5, 0.25, 0.0005, 0.0002
Sec. 7, Table 8	V	single	wall-clock

Alternatively, `make run-all` from `src/` executes the entire matrix over all four benchmarks and writes one log per experiment.